

Logistic regression text analysis explained simply ▾

08_ML_optimization_activity_blank.ipynb

IPYNB

08_ML_intro_activity_blank(1).ipynb

IPYNB

Can you explain what is going on here and what text components the logistic regression analyzes? What is the logic behind each group of code and what it does overall? Explain it to me in detail like I am 5

Viewed a file, ran a command >

Great, I've read both notebooks fully. Here's a thorough, plain-English breakdown of everything going on:

The Big Picture: What Are These Notebooks About?

Both notebooks are teaching you **machine learning on text** — specifically, how to take written reviews and have a computer automatically figure out whether they're positive, negative, or neutral. Think of it like teaching a robot to read and have feelings about sentences.

- **Notebook 1 (ML Intro)** → Uses **Yelp reviews**, trains a **Logistic Regression** model
- **Notebook 2 (ML Optimization)** → Uses **airline tweets**, trains **Decision Trees** and **Random Forests**

Notebook 1: The Intro Notebook (Yelp + Logistic Regression)

🧩 Block 1: Loading Tools

```
python  
  
import pandas as pd  
import re
```

```
from sklearn.linear_model import LogisticRegression
...
```

This is like getting your toolbox out before building furniture. Each `import` grabs a pre-built tool someone else wrote. `pandas` handles spreadsheet-like data, `sklearn` has all the ML algorithms, and `nltk` handles language stuff like stopwords.

Block 2: Text Cleaning Functions

python

```
def processtext(one_str, stop_list):
    no_stop = [tok for tok in wordpunct_tokenize(one_str) if tok not in stop_list]
    processed_string = " ".join([porter.stem(i.lower()) for i in no_stop if i.lower
```

Imagine you have a messy review: *"I was SO amazingly HAPPY eating there!!!"*

This function does three things:

1. **Removes stopwords** — throws out boring words like "I", "was", "the", "a" that don't carry meaning
2. **Lowercases everything** — so "Happy" and "happy" are treated the same
3. **Stems words** — chops words down to their root, so "running", "runs", "ran" all become "run"

The result is a stripped-down version of the review that only keeps the meaty, meaningful words.

Block 3: Loading Yelp Data

python

```
yelp = pd.read_pickle("../public_data/yelp_forML.pkl")
```

This loads a dataset of Yelp reviews that have already been **labeled** — meaning someone went through and tagged each review as, say, "good" (1) or "bad" (0). This label is what the model is trying to learn to predict.

```
python

vectorizer = CountVectorizer(lowercase=True)
dtm_sparse = vectorizer.fit_transform(list_of_strings)
```

Computers can't read words — they only understand numbers. So this step converts each review into a big row of numbers.

Imagine you have 3 reviews and the whole vocabulary is ["good", "bad", "food", "slow"]. Each review becomes a row:

Review	good	bad	food	slow
"good food"	1	0	1	0
"bad slow food"	0	1	1	1

This giant table is called a **Document-Term Matrix (DTM)**. Each row is one review, each column is a word, and the number is how many times that word appeared. This is the "bag of words" approach — the order of words is ignored; only their presence/count matters.

Block 5: Train/Test Split

```
python

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Before teaching the model, you hide 20% of the data in a "test vault." The model never sees this during training. Later, you check if it correctly predicts the labels on this hidden data — that's how you know if it really *learned* something or just memorized the answers.

Think of it like a student studying for an exam: the training data is the textbook, and the test data is the actual exam questions the student has never seen.

Block 6: The Logistic Regression Model (with L1/Lasso)

```
python
```

```
logit_lasso = LogisticRegression(penalty="l1", max_iter=100, C=0.01, solver='liblinear')
logit_lasso.fit(X_train, y_train)
```

This is the star of the intro notebook. Logistic regression looks at each word and asks: *"Does the presence of this word push the review toward positive or negative?"*

It assigns a **coefficient** (a score) to each word:

- A high positive score → word strongly suggests a *positive* review (e.g., "delicious", "amazing")
- A high negative score → word strongly suggests a *negative* review (e.g., "disgusting", "rude")

What text does it analyze? The cleaned, stemmed, stopwords-removed Yelp review text — converted into that word-count table (DTM). So it's looking at word frequencies across potentially thousands of vocabulary words.

L1 (Lasso) regularization is a discipline mechanism — it forces the model to be picky and push most word scores to zero, keeping only the most important words. Without it, the model would try to use every single word and overfit (memorize the training data rather than learning patterns).

C = 0.01 controls how strict that discipline is. A very small C = very strict (few words matter). A larger C = more relaxed (more words are used).

Block 7: Making Predictions and Evaluating

```
python

y_pred = logit_lasso.predict(X_test)
y_predprob = logit_lasso.predict_proba(X_test)
```

Now the model guesses labels for the hidden test reviews. You get two things:

- **predict** → hard answer: "this is positive" or "this is negative"
- **predict_proba** → soft answer: "I'm 83% sure this is positive"

Then the code manually calculates **precision** and **recall**:

- **Precision:** Of all the reviews I called positive, how many actually were? (Don't cry wolf)
 - **Recall:** Of all the truly positive reviews, how many did I catch? (Don't miss any)
-

Block 8: Interpreting the Model

python

```
las_coef = pd.DataFrame({'coef': logit_lasso.coef_[0], 'feature_name': ...})  
las_coef.sort_values(by='coef', ascending=False)
```

This looks inside the model's brain and says: *"What words did you find most important?"* It lists the top words with the highest positive and negative coefficients, then makes a bar chart. This is where you'd actually see things like "great", "love", "perfect" on the positive side and "awful", "wait", "cold" on the negative side.

Block 9: Comparing Different Hyperparameters

python

```
c_list = np.linspace(4, 0.0001, 5)
```

This section asks: *"What if we tried different levels of strictness (C values)?"* It trains 5 versions of the model with different C values and compares their precision scores to find the sweet spot.

Notebook 2: The Optimization Notebook (Airline Tweets + Decision Trees)

Block 1: Data Loading + Cleaning Tweets

The dataset is **airline tweets** labeled as positive, neutral, or negative — now it's 3 **categories** instead of 2. The cleaning step strips out `@usernames`, `#hashtags`, and URLs since those don't carry sentiment meaning (a tweet directed at `@united` doesn't tell us how the person *feels*). Those pieces get replaced with placeholder words like `USER` or `HASHTAG`.

Block 2: Vectorization (same idea, different data)

Same bag-of-words approach, but using `binary=True` — meaning instead of counting how many times a word appeared, it just marks 1 (present) or 0 (absent). This sometimes works better for short texts like tweets.

Block 3: Decision Tree

```
python  
  
dt_classifier = DecisionTreeClassifier(criterion='gini', max_depth=10, ...)
```

Instead of logistic regression, this uses a **decision tree** — literally a flowchart of yes/no questions about words.

Example flow:

- Does the tweet contain "thank"? → No → Does it contain "cancel"? → Yes → **Negative**

The tree is actually visualized in the notebook so you can see which words it checks at each branch. `gini` measures how "mixed up" a node is — a node where half the tweets are positive and half are negative is very impure; the model tries to create splits that separate them cleanly.

Block 4: Cross-Validation

```
python  
  
scores = cross_val_score(dt_classifier, X_train, y_train, cv=3)
```

Instead of one train/test split, this does **3 rounds** — each time, a different third of the data is the test set. This gives a more reliable average accuracy and helps detect if the model got lucky with one particular split.

Block 5: Confusion Matrix

A visual table showing where the model makes mistakes. For example, maybe it's great at identifying negative tweets but confuses neutral and positive ones. This is much more informative than a single accuracy number.

Block 6: Grid Search (Parameter Tuning)

```
python

param_grid = {'min_samples_split': [2,10], 'min_samples_leaf': [2,10]}
GridSearchCV(dt_classifier, param_grid, cv=3)
```

This systematically tries every combination of parameters and finds the best one. It's like trying all possible oven temperatures and bake times to find the perfect cookie.

Block 7: Random Forest

```
python

rf_classifier = RandomForestClassifier(n_estimators=10, criterion='gini', max_depth=
```

A Random Forest builds **many decision trees** (10 here) and takes a vote. Each tree is trained on a slightly different random slice of the data and uses a random subset of words. The idea: one tree might get lucky or unlucky, but 10 trees voting together are more reliable — like asking 10 doctors instead of 1.

Summary: What Text Does the Logistic Regression Actually Analyze?

In Notebook 1, the logistic regression analyzes **Yelp review text**, specifically:

- After removing English stopwords ("the", "is", "a", etc.)
- After stemming words to their root forms ("running" → "run")
- After converting every review into word-frequency counts

The model then learns which specific **word stems** are statistically associated with positive vs. negative reviews, and uses those word patterns to classify new reviews it's never seen before.

Can you alter the vectorization process of either notebook to fit a different scale other than good or bad? For example can you alter the process to

identify if someone is selling a product or just furthering their personal content?

Great idea! This is totally doable — you'd essentially be changing the **label** the model learns to predict, while the vectorization pipeline stays largely the same. The words become evidence for a *different* question.

Let me build a modified version of Notebook 1's pipeline, swapping out the Yelp sentiment labels for a "**commercial vs. personal**" classifier.

Viewed a file, created a file, read a file >

Here's your modified notebook! Here's what actually changed and why:

What's Different

The label is the only fundamental change. Instead of a Yelp `metadata_label` column with good/bad values, we now have a `label` column where `1 = commercial` and `0 = personal`. The model doesn't inherently "know" what commercial means — it just learns that certain words statistically appear more often alongside whichever label you assign. The label IS the definition.

One meaningful upgrade: bigrams (`ngram_range=(1,2)`). The original notebook only looked at single words. For this task, two-word phrases matter a lot — "link bio", "dm me", "affiliate link", "code save" are powerful commercial signals that disappear if you split them into individual words. This one parameter change makes the vectorizer capture those phrases as features.

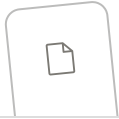
What Stayed Exactly the Same

The entire pipeline — `CountVectorizer`, train/test split, `LogisticRegression` with L1, precision/recall evaluation, coefficient interpretation — is structurally identical. That's the key insight: **the vectorization process doesn't care what you're classifying.** It just counts words. The label you attach to each text is what teaches the model what those word patterns *mean*.

To Make It Production-Quality

The demo dataset has 30 examples, which is enough to run the code but not enough to trust the results. For a real classifier you'd want:

- **500–5,000+ labeled posts** minimum
- **Clear labeling rules** written before you start (e.g., "any post with a promo code, affiliate link, or sponsored tag = commercial, regardless of tone")
- Optionally swap to `TfidfVectorizer` for larger datasets, which downweights words that appear everywhere and adds no signal



Commercial vs personal classifier
IPYNB

Download

How do you then train the new labels commercial and personal to know how two word phrases fit within this binary? Also between the decision trees and the logistic regression, which is better for a data analysis project on social media data observing the changing use of a specific hashtag over time by analyzing post captions? Should i do both or just one of the two?

Great questions — let me tackle both separately.

Question 1: How Does the Model Learn Two-Word Phrases?

The short answer: **you don't train it to understand phrases — the vectorizer just treats every phrase like its own brand new word, and the model learns from examples.**

Here's what actually happens step by step:

When you set `ngram_range=(1,2)`, the `CountVectorizer` scans every post and generates both single words *and* every pair of adjacent words. So the post "link in bio, use code SAVE20" doesn't just produce features like `link`, `bio`, `code` — it also produces `link in`, `in bio`, `use code`, `code save`. Each of those two-word phrases becomes its own column in the matrix, exactly like a single word would.

The model then sees thousands of training examples and notices: every time the column `link bio` has a 1, the label tends to be `commercial`. It has no concept of grammar or meaning — it's purely pattern-matching through repetition. The phrase is only useful if it shows up *enough times* in your training data with a *consistent label*. With 30 demo examples, `link bio` might appear twice and the model can barely learn from it. With

3,000 examples, it appears hundreds of times and the model becomes very confident about it.

This is also why **label consistency matters so much**. If you sometimes label a post as **personal** even though it says "link in bio" (maybe because the content felt genuine), the model gets contradictory signals and weakens that feature's coefficient.

Question 2: Decision Tree vs. Logistic Regression for Hashtag Research

For your specific project — tracking how use of a hashtag changes over time through caption analysis — here's the honest breakdown:

Use Logistic Regression. Don't bother with a Decision Tree.

Here's why, for each thing your project actually needs:

Interpretability of language. Your research question is about *how* the hashtag is being used — what words and themes are shifting. Logistic regression gives you a clean coefficient for every word, telling you exactly how strongly each one predicts each category. A decision tree gives you a flowchart of if/then rules, which is harder to summarize across thousands of posts and doesn't gracefully tell you "these 20 words define the commercial era of this hashtag."

Temporal analysis. You'll likely want to train on one time window and test on another, or compare coefficients across periods (e.g., "in 2021 the word *authentic* was a personal signal; by 2023 it became a commercial one"). Logistic regression coefficients are directly comparable across models. Decision tree structure is not — a tree trained on 2021 data looks completely different from one trained on 2023 data in ways that are hard to compare systematically.

Text data with many features. Your matrix will have thousands of word columns. Logistic regression handles high-dimensional sparse data very naturally. Decision trees tend to overfit badly in this setting unless you heavily tune them — they'll latch onto specific phrases that appear in training data but don't generalize.

Where a decision tree would actually win is if your features were structured variables like "number of hashtags used", "post length", "hour of day", "follower count" — concrete numeric or categorical things with clear thresholds. Trees are great at rules like "if follower

count > 10,000 AND post length < 50 words → commercial." That's a different kind of project than yours.

Should you do both anyway? Only if you have a specific reason to compare model performance numbers (like if this is a methods paper and you need a baseline). For a substantive research project focused on *what the language is doing*, logistic regression is the right tool and adding a decision tree would just be extra work without additional insight.

The one thing worth adding to your logistic regression is **TF-IDF weighting** instead of raw counts. For hashtag research across time, some words will appear in basically every post (like the hashtag itself) and TF-IDF automatically downweights those, making the remaining signals cleaner. Just swap `CountVectorizer` for `TfidfVectorizer` — everything else stays the same.

For the logistic regression can you show the specific locations where i should alter the vectorization process and where to add the TF-IDF weighting? Additionally, can you further explain the temporal analysis portion

Sure! Let me show you all three things visually.

remove / replace add no change needed

change 1 of 2 **Import block — add TfidfVectorizer**

STEP 1 — IMPORTS (TOP OF NOTEBOOK)

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

add One new import line right after the CountVectorizer import. You're just making TfidfVectorizer available to use — nothing else changes here.

change 2 of 2 **Vectorizer block — swap CountVectorizer for TfidfVectorizer**

STEP 4 — VECTORIZATION

```
vectorizer = CountVectorizer(
    max_features=500,
    binary=True,
    ngram_range=(1, 2)
)
```

```
vectorizer = TfidfVectorizer(
    max_features=500,
    ngram_range=(1, 2),
    sublinear_tf=True,
    min_df=2
)
```

```
X = vectorizer.fit_transform(df['clean_text']).toarray()
y = df['label'].values
```

```
feature_names = vectorizer.get_feature_names_out()
```

remove Delete the entire CountVectorizer block. The two lines below it (.fit_transform, .get_feature_names_out) stay exactly the same — TfidfVectorizer has the same interface.

add Replace with TfidfVectorizer using the same max_features and ngram_range. The two new parameters are explained below.

What the two new parameters do

sublinear_tf=True

Applies log-scaling to word counts. A word appearing 10 times isn't treated as 10× more important than one appearing once — it's treated as about 2× more important. Prevents very frequent words from dominating.

min_df=2

Only keeps words that appear in at least 2 posts. Filters out typos, one-off usernames, and noise words that appear in just one post and can't generalize to new data.

Everything else stays the same.

Train/test split, model fitting, evaluation, coefficient chart — zero changes. TfidfVectorizer produces a matrix with identical shape and column names; the model never knows the swap happened.

Now for the temporal analysis explanation — this is actually the most interesting methodological part of your project, so let me walk through it carefully.

Temporal Analysis: What It Means and How to Do It

The core idea is that you're not just asking "is this post commercial or personal?" — you're asking "**how did the ratio of commercial to personal posts change over time, and did the language itself shift?**" Those are two different questions and you need different approaches for each.

Question 1: Did the *proportion* of commercial posts change?

This is the simpler question. You train one model on a labeled sample, run it against all your posts, and then just count predictions grouped by time period.

```
python

# Assume df has a 'date' column and you've run predictions
df['predicted_label'] = model.predict(X_all)
df['month'] = pd.to_datetime(df['date']).dt.to_period('M')

# Proportion of commercial posts per month
df.groupby('month')['predicted_label'].mean().plot()
```

This gives you a line chart showing commercialization over time. If the hashtag started as a community tag and got co-opted by brands, you'd see the line rise.

Question 2: Did the *language* of commercial posts change?

This is where it gets more interesting and is the real value of the logistic regression approach. You train **separate models on different time windows** and compare the coefficients.

```
python

# Split your data by time period
df_early = df[df['year'] < 2021]
df_late  = df[df['year'] >= 2023]

# Fit a separate vectorizer+model on each period
# Then compare coefficients side by side
```

Now you can directly ask: what was a commercial signal word in 2020 that isn't anymore? What new words emerged as commercial signals by 2024? The coefficient table becomes a record of linguistic drift. A word like "authentic" might have had a negative coefficient (personal signal) in 2019 and a positive one (commercial signal) by 2022, which itself is a finding worth writing about.

The key methodological trap to avoid

If you use the same vectorizer fitted on all your data at once, words that only exist in later posts (new slang, new platform features like "collab") will bleed into the early model's vocabulary even though they didn't exist then. The cleaner approach is to **fit a fresh vectorizer on each time window independently**, which means the vocabulary itself is period-specific. The tradeoff is that coefficients are no longer directly comparable across models since the columns are different — so you'd compare by word, not by column index.

A practical middle ground is to fit one vectorizer on all your data (so the vocabulary is stable), but fit separate logistic regression models on each time slice. That way coefficients are directly comparable because every model has the same feature columns.

What "temporal" doesn't mean here

One thing worth clarifying: the model itself has no built-in sense of time. It doesn't learn that 2023 posts are different from 2019 posts on its own. You impose the temporal structure by deliberately splitting the data and training/evaluating across those splits. The model is always just doing the same binary classification job — you're the one drawing the timeline around it.

```
## function to create dtm
def create_dtm(list_of_strings, metadata):
    vectorizer = CountVectorizer(lowercase = True)
    dtm_sparse = vectorizer.fit_transform(list_of_strings)
    dtm_dense_named = pd.DataFrame(dtm_sparse.todense(),
    columns=vectorizer.get_feature_names_out())
    dtm_dense_named_withid = pd.concat([metadata.reset_index(),
    dtm_dense_named], axis = 1)
    return(dtm_dense_named_withid)
```

what does this do?

also what needs to change in the vectorization process to change the scale from good and bad to commercial and personal

What `create_dtm` does

This function takes your cleaned text and converts it into that word-count table (the Document-Term Matrix) we talked about earlier. Line by line:

`CountVectorizer(lowercase=True)` — creates the tool that will scan all posts and build a vocabulary. `lowercase=True` means "Hello" and "hello" get counted as the same word.

`vectorizer.fit_transform(list_of_strings)` — two things happen in one step. "Fit" means it reads all your posts and decides what the vocabulary is. "Transform" means it then counts how many times each vocabulary word appears in each post. The result is a compressed sparse matrix (just a memory-efficient way to store a table that's mostly zeros).

`pd.DataFrame(dtm_sparse.todense(), ...)` — converts that compressed matrix into a normal readable table with word names as column headers.

`pd.concat([metadata..., dtm_dense_named], axis=1)` — glues your metadata columns (like the label, the row ID, the original text) back onto the left side of the word-count table, so you don't lose track of which row belongs to which post.

The function returns one big combined table: metadata on the left, word counts on the right.

What actually needs to change for commercial vs. personal

Honestly, very little inside this function itself. The vectorizer doesn't know or care what your labels mean — it just counts words. The label lives in a completely separate column. Here's exactly what changes:

The function itself unchanged

ORIGINAL (YELP, GOOD/BAD)

```
def create_dtm(list_of_strings, metadata):
    vectorizer = CountVectorizer(
        lowercase=True
    )
    dtm_sparse = vectorizer
        .fit_transform(list_of_strings)
    dtm_dense_named = pd.DataFrame(
        dtm_sparse.todense(),
        columns=vectorizer
            .get_feature_names_out()
    )
    dtm_dense_named_withid = pd.concat(
        [metadata.reset_index(),
         dtm_dense_named], axis=1
    )
    return(dtm_dense_named_withid)
```

NEW VERSION (COMMERCIAL/PERSONAL)

```
def create_dtm(list_of_strings, metadata):
    vectorizer = TfidfVectorizer(
        ngram_range=(1, 2),
        sublinear_tf=True,
        min_df=2
    )
    dtm_sparse = vectorizer
        .fit_transform(list_of_strings)
    dtm_dense_named = pd.DataFrame(
        dtm_sparse.todense(),
        columns=vectorizer
            .get_feature_names_out()
    )
    dtm_dense_named_withid = pd.concat(
        [metadata.reset_index(),
         dtm_dense_named], axis=1
    )
    return(dtm_dense_named_withid)
```

The function body is identical except for swapping the vectorizer on line 2. The four lines that build and return the table are completely untouched — TfidfVectorizer produces a matrix with the same shape and interface as CountVectorizer.

The label column this is the real change

original column name	metadata_label → values: "good" / "bad"
new column name	label → values: 1 (commercial) / 0 (personal)
where it lives	in the metadata dataframe passed into the function
does the vectorizer touch it?	no – it only sees the text. the label rides along in metadata.

The vectorizer never sees your `list_of_strings`. Changing good/bad → commercial/personal has zero effect on how the word-count table is built.

Your label metadata dataframe `y =`, you'd 'label' — that's the
column just you pass in. `yelp_dtm[['metadata_label']]` change only
needs to When you that downstream
exist in the later run column update.
name to

Where you call the function update column names

ORIGINAL CALL

```
yelp_dtm = create_dtm(
    yelp['process_text'],
    yelp[['metadata_label',
          'metadata_rowid',
          'process_text',
          'raw_text']]
)
```

NEW CALL

```
df_dtm = create_dtm(
    df['clean_text'],
    df[['label',
          'metadata_rowid',
          'process_text',
          'raw_text']]
)
```

Same function call structure. You're just passing in your new dataframe with its new column names. The function doesn't care what they're called — it concatenates whatever metadata you give it.

The key mental model here is that the function has two completely separate lanes running through it. The `list_of_strings` lane is pure text — the function counts words and has no awareness of what any label means. The `metadata` lane is just cargo — it gets glued onto the side of the word table and passed through untouched. Changing your classification goal from good/bad to commercial/personal only affects what values you put in the metadata lane. The word-counting machinery in the middle doesn't change at all.

```
porter = PorterStemmer()
list_stopwords = stopwords.words("english")
what does this do?
```

These two lines are just setup — they initialize two tools you'll use during text cleaning.

`PorterStemmer()` creates a stemmer object. Stemming is the process of chopping words down to their root form. For example:

- "running", "runs", "runner" → all become `run`
- "selling", "seller", "sold" → all become `sell`
- "commercial", "commercially" → both become `commerci`

The stemmer doesn't produce real English words — it's just a mechanical chopper following a ruleset. The point isn't readability, it's that the model now treats all those word variations as the same feature instead of counting them separately.

`stopwords.words("english")` loads a pre-built list of about 180 common English words that carry almost no meaning on their own — things like "the", "is", "a", "in", "at", "we", "their". These get filtered out before vectorization because they appear in nearly every post regardless of whether it's commercial or personal, so they add noise without adding signal.

Neither line actually *does* anything to your text yet. Think of them like getting a knife out of the drawer and a strainer out of the cabinet — you're just making the tools ready. The actual chopping and filtering happens later inside the `processtext` function when it calls `porter.stem()` and checks each word against `list_stopwords`.

One thing worth knowing for your specific project: the standard English stopwords list was built for general text. For social media hashtag research you may want to customize it. Words like "just", "really", and "so" are on the default list, but in caption analysis they might actually signal something — "just dropped", "so excited to share" lean commercial, while "just feeling" or "so grateful today" lean personal. You can add or remove words from the list like any normal Python list before passing it into `processtext`.

```
porter = PorterStemmer() list_stopwords = stopwords.words("english")
tcm_df_creator['process_text'] = [processtext(one_review, stop_list =
list_stopwords) for caption in post['text_clean']] tcm_creator_dtm =
create_dtm(tcm_df_creator['process_text'],
tcm_df_creator[['metadata_label', 'metadata_rowid', 'process_text',
'raw_text']])
how do you adjust the metadata label in this to fit the binary commerical vs
personal in a logistic regression
```

There's actually a bug in this code before we even get to the label question. Look at this line:

```
python
```

```
tcm_df_creator['process_text'] = [processtext(one_review, stop_list = list_stopwords
                                             for caption in post['text_clean'])]
```

The loop variable is `caption` but the function receives `one_review` — those two names don't match, so Python would throw a `NameError`. Both need to be the same variable name. Here's the corrected and relabeled version of the whole block:

```
python

porter = PorterStemmer()
list_stopwords = stopwords.words("english")

# fix: loop variable and function argument now match
tcm_df_creator['process_text'] = [processtext(caption, stop_list=list_stopwords)
                                  for caption in tcm_df_creator['raw_text']]

# fix: metadata_label column must contain 1 (commercial) or 0 (personal)
tcm_creator_dtm = create_dtm(
    tcm_df_creator['process_text'],
    tcm_df_creator[['label',          # renamed from metadata_label
                    'metadata_rowid',
                    'process_text',
                    'raw_text']]
)
```

But the most important change is upstream of this block entirely — your dataframe needs to actually *have* a `label` column with the right values before you call any of this. That's where the real work happens:

```
python

# This must exist in your dataframe BEFORE the block above runs.
# You either load it from a CSV you labeled yourself,
# or you assign labels manually like this:

tcm_df_creator['label'] = [1, 0, 1, 1, 0, ...] # 1=commercial, 0=personal
                                           # one value per row
```

The three things that changed and why:

The loop variable bug is fixed — `caption` is now used consistently on both sides of the `for` loop. The metadata column `metadata_label` is renamed to `label` containing `1/0` integers instead of string values like `"good"/"bad"` — logistic regression expects numbers, not words. And the text source is `tcm_df_creator['raw_text']` rather than `post['text_clean']`, since you're pulling from your own dataframe rather than a separate object.

where do you adjust the label scale, which part of the code and how

The label lives in exactly one place — your dataframe, before any modeling code runs. Everything else (the vectorizer, the DTM, the logistic regression) just reads whatever values are already in that column. Here's where it sits in the full pipeline:

where you set the label

where the label is read

unaffected by label

STEP 1 — SET YOUR LABEL HERE

SET LABEL

```
tcm_df_creator['label'] = [1, 0, 1, 0, ...]
# 1 = commercial, 0 = personal
# one integer per row, in the same order as your posts
# this is the ONLY place the scale is defined
```

This is the only line you write to define what commercial and personal mean. The numbers you put here are what the model learns from.



STEP 2 — TEXT CLEANING

UNAFFECTED

```
tcm_df_creator['process_text'] = [processtext(caption, stop_list=list_stopwords)
                                  for caption in tcm_df_creator['raw_text']]
```

Only touches the text column. Knows nothing about labels.



STEP 3 — BUILD DTM

READS LABEL

```
tcm_creator_dtm = create_dtm(
    tcm_df_creator['process_text'],
    tcm_df_creator[['label', 'metadata_rowid',
                    'process_text', 'raw_text']]
)
```

The label column is carried along as metadata. The vectorizer still only processes the text — it just needs the label present so it ends up in the final table.



STEP 4 — SPLIT X AND Y

READS LABEL

```
X = tcm_creator_dtm[tcm_creator_dtm.columns[5:]]
y = tcm_creator_dtm[['label']]
```

This is where the label column is separated out and handed to the model as the thing to predict. No changes needed here beyond using the new column name.



STEP 5 — TRAIN/TEST SPLIT

UNAFFECTED

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Splits whatever is in X and y. Doesn't care what the label values mean.



STEP 6 — LOGISTIC REGRESSION

UNAFFECTED

```
logit_lasso = LogisticRegression(penalty='l1', C=0.01, solver='liblinear')
logit_lasso.fit(X_train, y_train)
```

Receives numbers (1s and 0s) from `y_train`. Works identically whether those numbers mean good/bad or commercial/personal.

So the full answer is: you adjust the label scale in step 1 only, by writing integers into a column in your dataframe. Every step after that just reads whatever is already there — none of them need to be rewritten when you change the scale.

The practical question this raises is how you actually get those 1s and 0s into your dataframe in the first place. There are three realistic ways depending on your situation:

If you're labeling by hand, you export your posts to a CSV, add a `label` column yourself in Excel or Google Sheets, put 1 or 0 next to each post, save it, and load it back with `pd.read_csv()`. This is the most reliable approach for a research project because you control exactly what the labels mean.

If your data already has a usable signal column — like a follower count threshold, a verified account flag, or a column that tracks whether a post contains a URL — you can derive the label programmatically:

python

```
# example: posts with URLs and hashtags are probably commercial
tcm_df_creator['label'] = ((tcm_df_creator['has_url'] == True) &
                           (tcm_df_creator['hashtag_count'] > 3)).astype(int)
```

If you have no labels at all and can't create them manually, you'd need a different approach entirely — unsupervised methods like topic modeling — but that's a separate conversation. Logistic regression specifically requires that you already know the answer for at least a few hundred posts before it can learn to predict the rest.

but this does not indicate anything about personal or commercial? Where do the values in the list come from?

You're exactly right — and this is the most important thing to understand about supervised machine learning.

The values come from you. A human has to create them.

The model has no idea what "commercial" or "personal" means until you sit down and manually read through your posts and assign a 1 or 0 to each one. That human-labeled data is called your **training data**, and it is the entire foundation the model learns from. There is no shortcut around this step.

Here's what that actually looks like in practice:

```
python

# You export your posts
tcm_df_creator[['metadata_rowid', 'raw_text']].to_csv('posts_to_label.csv', index=False)
```

You open that CSV in Excel or Google Sheets and it looks like this:

metadata_rowid	raw_text	label
001	"Use code SAVE20, link in bio!"	1
002	"Spent the morning hiking with my dog"	0
003	"AD: sponsored by VPN service"	1
004	"I've been thinking about how much I've changed"	0

You fill in that `label` column yourself, row by row. Then you load it back:

```
python

labels = pd.read_csv('posts_to_label.csv')
tcm_df_creator = tcm_df_creator.merge(labels[['metadata_rowid', 'label']],
                                       on='metadata_rowid')
```

Now `tcm_df_creator['label']` actually exists with real values, and the pipeline we discussed works.

The reason this matters so much for your specific project is that **your labeling decisions define what the model learns**. Before you label a single post you need to write down a clear rule for yourself, something like:

- Label 1 (commercial) if: the post contains a promo code, affiliate link, brand partnership disclosure, product price, or explicit call to action like "buy", "shop", "DM me"
- Label 0 (personal) if: the post shares an opinion, experience, emotion, or creative work with none of the above

If you're inconsistent — sometimes labeling a subtle sponsored post as personal because the writing felt genuine — the model learns that inconsistency and gets confused. The quality of your labels is more important than any parameter you set in the model itself.

For a research project on hashtag usage you probably only need to manually label **300-500 posts** to get a working model. You label those, train on them, and then the model predicts labels for the remaining thousands automatically.

is there no alternative way to do this?

Yes, there are two real alternatives that don't require manual labeling.

Option 1: Rule-based labeling (most practical for your project)

Instead of reading posts by hand, you write explicit rules that assign the label automatically based on patterns in the text. No model needed for the labeling step — just logic.

```
python

import re

def label_post(text):
    text_lower = text.lower()

    commercial_signals = [
        r'link in bio', r'swipe up', r'use code', r'promo code',
```

```

    r'discount', r'affiliate', r'sponsored', r'#ad\b', r'#spon\b',
    r'shop now', r'buy now', r'dm me', r'click the link',
    r'\$\d+', r'% off', r'free shipping', r'collab'
]

for pattern in commercial_signals:
    if re.search(pattern, text_lower):
        return 1 # commercial

return 0 # personal by default

```

python

```
tcm_df_creator['label'] = tcm_df_creator['raw_text'].apply(label_post)
```

The label column now fills itself. You don't read a single post manually. The tradeoff is that your rules only catch what you explicitly list — a subtle influencer post that says "obsessed with this product" with no hashtag or promo code would slip through as personal. But for a research project tracking hashtag use over time, obvious commercial signals are probably exactly what you want to measure anyway.

Option 2: Use an LLM to label for you (most powerful)

You send each post to an AI model and ask it to assign the label. This is called **LLM-assisted annotation** and it's increasingly common in research.

python

```

import anthropic

client = anthropic.Anthropic(api_key="your_key_here")

def llm_label(text):
    response = client.messages.create(
        model="claude-opus-4-6",
        max_tokens=10,
        messages=[{
            "role": "user",
            "content": f"""Classify this social media post as commercial or personal.

Commercial = selling a product, promoting a brand, affiliate content,

```

sponsored post, driving traffic, monetizing an audience.

Personal = sharing an opinion, experience, emotion, or creative content with no commercial motive.

Reply with ONLY the number 1 (commercial) or 0 (personal). Nothing else.

```
Post: {text}"""
    }]
)
result = response.content[0].text.strip()
return int(result) if result in ['0', '1'] else None

tcm_df_creator['label'] = tcm_df_creator['raw_text'].apply(llm_label)
```

This is slower and costs money per API call, but it handles nuance that rules miss — a post saying "obsessed with this, you need it in your life  " would get caught as commercial even without an explicit promo code, because the model understands intent.

Which should you use?

For your hashtag research project specifically, Option 1 is probably the right call. Here's why:

- Your research question is about *observable commercial behavior* — explicit selling language, disclosures, calls to action. Rule-based labeling captures exactly that and makes your methodology transparent and reproducible.
- Option 2 is better when the distinction is subtle and requires reading intent, like distinguishing sincere reviews from astroturfing.
- Both can be combined — use rules first, then send only the ambiguous cases (posts that almost matched but didn't quite) to the LLM for a second opinion.

Neither approach fully replaces human judgment, but for a well-defined research question like yours, rules get you surprisingly far with much less effort.

```
porter = PorterStemmer()
list_stopwords = stopwords.words("english")
tcm_df_creator['process_text'] = [processtext(caption, stop_list =
list_stopwords)
```

```

        for caption in post['text_clean']]
tcm_creator_dtm = create_dtm(tcm_df_creator['process_text'],
tcm_df_creator[['metadata_label', 'metadata_rowid',
                'process_text', 'raw_text']])

where does this tcm_df_creator['label'] =
tcm_df_creator['raw_text'].apply(label_post) go in the code above

```

It goes right at the top, before anything else runs, because the label column needs to exist in the dataframe before it gets passed into `create_dtm`.

```

python

porter = PorterStemmer()
list_stopwords = stopwords.words("english")

# STEP 1 – create the label column first
tcm_df_creator['label'] = tcm_df_creator['raw_text'].apply(label_post)

# STEP 2 – clean the text
tcm_df_creator['process_text'] = [processtext(caption, stop_list=list_stopwords)
                                for caption in tcm_df_creator['raw_text']]

# STEP 3 – build the DTM, now passing 'label' instead of 'metadata_label'
tcm_creator_dtm = create_dtm(
    tcm_df_creator['process_text'],
    tcm_df_creator[['label',                        # changed from 'metadata_label'
                    'metadata_rowid',
                    'process_text',
                    'raw_text']]
)

```

Two things changed beyond just adding the new line. The label line uses `raw_text` as the source because that's the original unprocessed text — you want to run your commercial/personal rules against the full original caption, not the stemmed cleaned version where words like "sponsored" might have been chopped or removed. And `metadata_label` in the `create_dtm` call becomes `label` to match the column you just created.

The order matters here because Python runs top to bottom — if you tried to reference `tcm_df_creator['label']` inside `create_dtm` before that column existed, you'd get a

`KeyError`.

what does `metadata_rowid` correlate to? The label in the good vs bad binary

No, `metadata_rowid` has nothing to do with the label. They're completely separate things.

`metadata_rowid` is just a **unique ID number for each row** — think of it like a receipt number. It's purely for tracking which post is which. It tells you nothing about the content or category of the post.

Think of it like a spreadsheet:

metadata_rowid	raw_text	label
001	"Use code SAVE20, link in bio!"	1
002	"Spent the morning hiking"	0
003	"AD: sponsored by VPN"	1

`metadata_rowid` is just the leftmost ID column. `label` is the meaning column. They're independent — the ID doesn't tell you anything about whether a post is commercial or personal, it just makes sure you can always trace a prediction back to the exact original post that produced it.

It becomes useful later when your model makes a prediction and you want to look up which specific post it predicted on. Without a row ID, if you have 10,000 posts that all look similar, you'd have no reliable way to find post number 7,842 again after the DTM has scrambled the original order.

In the original Yelp notebook `metadata_rowid` played the same role — it was never part of what the model learned from, it was just a bookkeeping number so you could always trace results back to the original review. That purpose is identical in your version.



